CasMotif: Information Cascade Prediction via Causal Temporal Motifs and Attentive Sequence Models

Varun S[1], Kavin Karthic M[1], Aswath Siddharth R[1], Darshan R[1], Harish Krishna B[1], and Dayanand V[2]

[1] Department of Computer Science and Engineering,
Amrita Vishwa Vidyapeetham, Coimbatore, India
{cb.sc.u4cse23157, cb.sc.u4cse23161, cb.sc.u4cse23109, cb.sc.u4cse23115, cb.sc.u4cse23323}@cb.students.amrita.edu
[2] Assistant Professor, School of Computing,
Amrita Vishwa Vidyapeetham, Coimbatore, India
v_dayanand@cb.amrita.edu

Abstract. Predicting how widely online content will spread from its earliest interactions remains a challenging problem in social network analysis. Many deep-learning approaches capture cascade structure only indirectly, through random walks, graph convolution, or temporal graph attention, so the local interaction patterns that emerge as a cascade develops are rarely represented explicitly. We introduce **CasMotif**, which describes an early cascade as an ordered sequence of motif-annotated events. As each retweet edge arrives, a streaming component records the patterns it completes—a sibling fork, a deep chain, a reciprocal dyad, a directed three-cycle, or a plain reply—computed on the graph as it existed immediately *before* the edge was added, together with structural position, inter-arrival time, motif-formation latency, and user reactivation. A bidirectional GRU with additive attention pooling (49,824 parameters) maps this 15-dimensional event sequence to the logarithm of the future popularity increment. On the Sina Weibo benchmark with the DeepHawkes preprocessing, CasMotif achieves a three-seed mean MSLE of 1.985 ($\log_2$) at the standard one-hour window. This is statistically indistinguishable from CasSubTS on the same splits and seeds (1.986; paired t-test $p = 0.93$) while using 3.1 times fewer parameters, and 18.7% below the 2.441 reported for DeepHawkes. With only 15 minutes of activity, CasMotif reaches an MSLE of 2.441 and $R^2 = 0.576$, a 57.6% error reduction over a training-mean predictor. Replacing recurrent attention with mean or max pooling increases error by 13.7% and 19.2% in this setting.

Keywords: Information cascades · Popularity prediction · Temporal motifs · Early prediction · Sequence models · Social networks

1 Introduction

Content on social platforms spreads through information cascades: users share a post, and their activity can prompt further users to share it. Understanding how these cascades develop, and predicting their eventual reach, has become an important topic in social network analysis [1]. Popularity forecasts support applications such as viral marketing [2], where influence can be deliberately seeded across a network [3], and the study of misinformation, which has been shown to spread farther, faster, deeper, and more broadly than true news on Twitter [4]. Researchers have therefore investigated a wide range of signals and models for predicting the popularity of content on microblogging platforms [5].

The extent to which cascade popularity can be predicted is still debated. Experiments in an artificial cultural market found that social influence can increase both inequality and unpredictability in success [6], while analyses of Twitter cascades suggest that practical limits constrain how accurately their final sizes can be forecast [7]. Other work has found that early observations can nevertheless provide useful signals. For example, Cheng et al. showed that temporal and structural characteristics help predict the growth of Facebook photo cascades, and that early breadth can be more informative than depth [8]. The popularity of online content early in its lifetime has also been found to correlate with later popularity after logarithmic transformation [9]. Shulman et al. reported that temporal features account for much of the predictive power of early-adopter models, whereas the relationship between network features and popularity varies across domains [10]. Taken together, these findings suggest that the way early cascade structure is represented matters—particularly when only a small number of initial interactions are available.

Existing approaches can be grouped into three broad families. *Feature-based* methods use manually designed temporal, structural, and community-level descriptors of the observed cascade [8,11,13]. *Generative* approaches model reshare arrivals as a point process, often using self-exciting Hawkes processes [15,18] or reinforced Poisson processes [17]. *Deep-learning* methods instead learn representations from cascade data. DeepCas samples random-walk sequences from the cascade graph [23], DeepHawkes combines recurrent learning with interpretable Hawkes-process factors [24], and CasCN applies graph convolution to sequences of sub-cascade graphs [25]. More recent approaches model hierarchical uncertainty [26], combine subgraph sampling with multi-head graph attention [27], link cascades through continuous-time interactions [28], or apply hierarchical temporal graph attention [29].

Network science offers another useful perspective: motifs are recurring local connection patterns that can reveal structure not captured by individual edges alone [30,31,32,33]. Temporal motifs also account for the order and timing of interactions [31,32]. Yet many deep cascade predictors absorb local structure into learned graph or sequence embeddings rather than recording explicitly which patterns are formed as each new edge arrives [23,25,27]. This leaves two questions. Can a causal, event-by-event record of local motifs provide a useful structural signal for popularity prediction? And how much predictive informa-

tion remains when the observation window is substantially shorter than those commonly used in benchmark evaluations?

We investigate these questions with **CasMotif**. Its main contributions are:

- **Causal temporal-motif event representation.** Each retweet edge is described by motif indicators for a sibling fork, deep chain, reciprocal dyad, directed three-cycle, or plain reply. These indicators are computed on the cascade graph *before* the new edge is inserted. Structural position, inter-arrival timing, motif-formation latency, and user-reactivation features are included in a 15-dimensional event vector, avoiding look-ahead.
- **A compact sequence model.** A bidirectional GRU, additive attention pooling, and a small regression head map the event sequence to the $\log_2$ of the future popularity increment. The model has 49,824 trainable parameters and does not require graph-level message passing at prediction time.
- **Evaluation with a very short observation window.** Using the public Sina Weibo benchmark and the preprocessing supplied with DeepHawkes, we evaluate predictions from the first 15 minutes of cascade activity. This is shorter than the windows we found reported for this dataset; the shortest identified in the reviewed work was 30 minutes [29].
- **Empirical comparison and ablation.** At the standard one-hour window, CasMotif obtains a three-seed mean MSLE of 1.985, comparable to CasSubTS [27] (1.986, $p = 0.93$) on identical splits while using fewer parameters. At 15 minutes, it achieves $R^2 = 0.576$. Ablations examine the contribution of recurrent attentive aggregation relative to order-independent pooling.

2 Related Work

2.1 Feature-Based Popularity Prediction

Early work treated popularity prediction as a supervised-learning problem using manually engineered features of a cascade's initial activity. Szabo and Huberman showed that the logarithm of early popularity on Digg and YouTube can predict later popularity [9]. Cheng et al. combined content, user, structural, and temporal information to predict whether Facebook photo cascades would double in size [8]. Weng et al. demonstrated that the distribution of early adopters across network communities can help identify viral content [11] and later showed the value of community-structure features for predicting meme success [12]. Bao et al. examined the structural characteristics of early participants on Sina Weibo [13]. Goel et al. introduced *structural virality* to distinguish broadcast-like diffusion from spread across multiple generations, finding that it was generally low and largely unrelated to cascade size on Twitter [14]. A recurring limitation is that features predictive in one domain often fail to transfer to others [10].

2.2 Generative and Point-Process Models

Generative approaches model the arrival of reshares directly. The Hawkes process [15] has been central to this line of work. Crane and Sornette used response functions to characterise endogenous and exogenous bursts in YouTube popularity [16], while Shen et al. modelled popularity dynamics with reinforced Poisson processes that incorporate fitness, temporal relaxation, and reinforcement [17]. SEISMIC predicts final retweet counts from reshare times and node degrees using a self-exciting point process, without requiring model training [18]. Other work has modelled retweet dynamics on microblogging platforms [19], connected exogenous promotion and endogenous popularity through Hawkes intensity processes [20], or combined one-hop sub-cascades into larger-scale predictions using networked Weibull regression [21]. Mishra et al. studied ways to combine feature-driven and point-process approaches [22]. These methods offer interpretable accounts of diffusion, although DeepHawkes argued that their predictive performance can be surpassed by learned representations [24].

2.3 Deep Cascade Representation Learning

DeepCas introduced an end-to-end approach that represents a cascade graph through random-walk node sequences and aggregates them with a GRU and attention [23]. DeepHawkes models retweet paths recurrently while retaining user-influence, self-excitation, and time-decay factors from Hawkes processes [24]. Its release also provides the Sina Weibo dataset used here. CasCN learns the structural and temporal evolution of a cascade by applying graph convolution to a sequence of sub-cascade graphs, followed by a recurrent network [25]. CasFlow models hierarchical cascade structure and uncertainty in propagation [26], while CCGL learns cascade representations through contrastive self-supervision [34]. CasSeqGCN combines network structure with temporal sequences [35], and Sun et al. incorporate explicit time embeddings into a cascade-attention model [36]. CasSubTS combines temporal and structural signals using subgraph sampling and multi-head attention [27]. CTCP models dependencies between cascades through user–cascade and user–user interactions in continuous time [28]. HierCas uses the full cascade graph, time-aware node embeddings, graph attention, and hierarchical pooling [29]. A related line of work predicts which individual user will be infected next, rather than forecasting the final size of the entire cascade [37]. Zhou et al. provide a broader survey of information-cascade analysis [1].

2.4 Network Motifs and Temporal Graph Learning

Milo et al. described network motifs as recurring patterns of interconnection that appear more often than expected in randomized networks [30]. Kovanen et al. extended the idea to temporal networks, where the order and timing of events are part of the pattern [31], and Paranjape et al. formalised δ-temporal motifs and proposed efficient counting algorithms [32]. Benson et al. showed

that higher-order clustering based on motifs can reveal organization that is not apparent from individual edges [33].

Temporal structure is also central to representation learning for dynamic graphs [38]. JODIE learns coupled recurrent embedding trajectories for temporal interaction networks [39], TGAT combines functional time encodings with self-attention for inductive temporal-graph learning [40], and TGN maintains node memories that are updated as interaction events occur [41].

CasMotif connects these areas. Like feature-based methods, it uses interpretable structural and temporal descriptors. However, rather than summarising an entire cascade with aggregate features, it calculates descriptors for each event using only the graph state available at that point in time. Like deep cascade models, it is trained end-to-end for popularity prediction, but it uses a recurrent encoder over motif-annotated events instead of graph-level message passing.

3 Problem Formulation

We follow the incremental-popularity formulation used for the DeepHawkes benchmark [24].

Definition 1 (Information cascade). *Let a message m be published by user u_0 at time t_0. Its* cascade *is the time-ordered collection $C_m = \{(p_j, t_j)\}_{j=0}^{N_m}$. Each propagation path $p_j = (u_0, u_1, \ldots, u_{k_j})$ records the sequence of users from the original author to the user who reshared the message. The timestamp $t_j \geq 0$ denotes elapsed time since publication, and $j = 0$ represents the original post.*

Definition 2 (Cascade graph and causal state). *Each reshare with $k_j \geq 1$ creates a directed edge $e_j = (u_{k_j-1} \to u_{k_j}, t_j)$, linking the immediate parent to the resharing user. At time t, the cascade graph is $G_t = (V_t, E_t)$, where $E_t = \{e_j : t_j \leq t\}$ contains the edges whose timestamps are no later than t. For an observed edge e_i, G_{i^-} denotes the graph containing all edges inserted* before *e_i. This is the causal state used to compute the event's features.*

Definition 3 (Observed size and popularity increment). *Let T_o be the observation window and $T_p > T_o$ the prediction horizon. The observed size is $n_m(T_o) = |\{j : t_j < T_o\}|$, and the eventual size is $N_m(T_p) = |\{j : t_j < T_p\}|$. Both counts include the original post. The target is the future increment $\Delta P_m = N_m(T_p) - n_m(T_o)$.*

Task. Given only reshares observed in $[0, T_o)$, the model predicts the log-transformed increment:

$$\hat{y}_m = f\big(\{(p_j, t_j) : t_j < T_o\}\big) \approx y_m = \log_2(\Delta P_m + 1). \tag{1}$$

The primary metric is mean squared logarithmic error in base 2:

$$\text{MSLE} = \frac{1}{M} \sum_{m=1}^{M} \big(\log_2(\hat{\Delta P}_m + 1) - \log_2(\Delta P_m + 1)\big)^2. \tag{2}$$



Fig. 1. Overview of CasMotif. Each observed reshare edge is described using the cascade graph as it existed immediately before that edge was added (G_{i^-}). The resulting event sequence is encoded by a bidirectional GRU, summarised by additive attention pooling, and mapped to the log popularity increment.

The reported comparison follows the $\log_2$ convention used in the CasSubTS results [27]. MSLE values expressed in natural-log units are smaller by a constant factor of $(\ln 2)^2 \approx 0.48$, so the logarithm base is specified whenever a value is reported. Unless stated otherwise, the prediction horizon is $T_p = 24$ hours, and the observation window T_o is either 15 minutes or one hour.

4 The CasMotif Framework

CasMotif consists of two stages (Fig. 1). First, a *causal motif-and-state engine* converts the observed reshare stream into an ordered sequence of 15-dimensional event vectors (Sect. 4.1, 4.2). Second, an *attentive recurrent encoder* processes this sequence and predicts the log-transformed future increment (Sect. 4.3).

4.1 Causal Motif and State Engine

The engine processes reshares in chronological order over $[0, T_o)$ while maintaining a directed graph that is updated incrementally. Its state includes each node's outgoing adjacency list, a hash set of existing edges for constant-time edge checks, node-appearance histories, node depths, the first-seen parent of each node, and a running descendant count for every node.

For every incoming edge $e_i = (u \to v, t_i)$, all features are computed from the state *before* the edge is inserted. Only after feature extraction does the engine add the edge and update its state (Algorithm 1). An event therefore never sees its own effect, any later event, or anything beyond the observation window.

Motif indicators. Let $\mathcal{N}^+(\cdot)$ denote the outgoing neighbours in G_{i^-}, and let d_i be the depth of v along its propagation path. The engine checks five patterns:

$$\begin{aligned}
\textsc{Fork}_i &= \mathbf{1}\big[\,|\mathcal{N}^+(u)| \geq 1\,\big],\\
\textsc{Chain}_i &= \mathbf{1}\big[\,d_i \geq 3\,\big],\\
\textsc{Dyad}_i &= \mathbf{1}\big[\,(v \to u) \in G_{i^-}\,\big],\\
\textsc{Cycle}_i &= \mathbf{1}\big[\,\exists w \in \mathcal{N}^+(v) : (w \to u) \in G_{i^-}\,\big],\\
\textsc{Reply}_i &= \mathbf{1}\big[\,\neg(\textsc{Fork}_i \vee \textsc{Chain}_i \vee \textsc{Dyad}_i \vee \textsc{Cycle}_i)\,\big].
\end{aligned} \tag{3}$$

A *fork* indicates that the parent already has a child, so the new edge creates another branch from that parent. A *chain* indicates that the edge extends a

Algorithm 1 Causal event-sequence construction for one cascade

Require: observed reshares $\{(p_j, t_j) : t_j < T_o\}$ sorted by t_j
Ensure: event matrix $X \in \mathbb{R}^{K \times 15}$

initialise empty graph state; $t_{\text{prev}} \leftarrow 0$; $X \leftarrow [\,]$
for $j = 0, 1, \ldots$ in time order **do**
  if p_j is the original post **then** register u_0 at depth 0; $t_{\text{prev}} \leftarrow t_j$; **continue**
  end if
  $u \leftarrow$ parent of the resharing user in p_j; $v \leftarrow$ resharing user; $d \leftarrow |p_j| - 1$
  $\mathbf{m} \leftarrow$ motif indicators of $(u \to v)$ on the *current* state ▷ Eq. (3)
  compute Δt^M, T^{idle}, I^{react}, out-degree and descendant count of u from the current state
  insert $(u \to v, t_j)$; update histories, depths and ancestor descendant counts
  append $[\mathbf{m}, d, \ldots, t_j - t_{\text{prev}}]$ to X; $t_{\text{prev}} \leftarrow t_j$
end for
return X

path to a depth of at least three. A *dyad* occurs when the new edge closes a reciprocal pair, while a *cycle* occurs when it completes a directed three-cycle $u \to v \to w \to u$. Since a user can reshare more than once and re-enter a cascade, dyads and cycles are possible. The four higher-order indicators can be active at the same time, so they form a multi-hot feature block. The Reply indicator is active only when none of the other four patterns applies.

Computational cost. Fork, chain, and dyad checks take $O(1)$ time. The cycle check takes $O(|\mathcal{N}^+(v)|)$, and updating descendant counts requires walking through the ancestors of u, taking $O(d_i)$. Thus, processing a cascade with K observed edges takes $O\big(\sum_{i=1}^{K}(|\mathcal{N}^+(v_i)| + d_i)\big)$ time, with memory linear in the number of nodes and edges.

4.2 Event Features

Each event is represented by 15 features (Table 1). In addition to the five motif indicators, the features describe four aspects of cascade development:

- **Structural position:** the new node's depth, the parent's out-degree and descendant count, and the current size of the cascade graph.
- **Timing:** elapsed time since the original post and the interval since the previous event.
- **Motif-formation latency:** $\Delta t_i^M = t_i - \min \mathcal{H}(u)$, or the time elapsed since the parent first appeared. This indicates how quickly the local structure has formed.
- **User reactivation:** for a user v with earlier appearances $\mathcal{H}(v)$, idle time is $T_i^{\text{idle}} = t_i - \max \mathcal{H}(v)$. A dormancy indicator $I_i^{\text{react}} = \mathbf{1}[T_i^{\text{idle}} > 300\,\text{s}]$ is set to one if the idle time exceeds 300 seconds. For a user appearing for the first time, idle time is set to t_i and the reactivation indicator is zero.

Table 1. Fifteen event-level features. Features are calculated from G_{i^-} unless noted. For the scaled numerical features, $\log(1+x)$ is applied before standardisation using statistics fitted only on the training split.

#	Feature	Definition	Scaled
0–4	Motif block	Reply, Fork, Chain, Dyad, Cycle (Eq. 3)	no
5	Tree depth	depth d_i of the new node v	yes
6	Parent out-degree	$\lvert\mathcal{N}^+(u)\rvert$	yes
7	Node count	$\lvert V\rvert$ after inserting e_i	yes
8	Time from root	t_i	yes
9	Reactivation flag	I_i^{react}	no
10	Idle time	T_i^{idle}	yes
11	Parent descendants	number of edges in the sub-cascade rooted at u	yes
12	Motif latency	Δt_i^M	yes
13	Cumulative prior	number of observed entries preceding e_i	yes
14	Inter-arrival gap	$t_i - t_{i-1}$	yes

4.3 Attentive Recurrent Encoder

Let $X = [\mathbf{x}_1, \ldots, \mathbf{x}_K]^\top \in \mathbb{R}^{K\times 15}$ denote the scaled event sequence for one cascade. Each event is first passed through layer normalisation [42]. The sequence is then processed by a bidirectional recurrent network [43] built from gated recurrent units (GRUs) [44]. With hidden size $D = 64$ in each direction, the representation for event i combines information from earlier and later events within the observed sequence:

$$\mathbf{h}_i = \left[\overrightarrow{\mathrm{GRU}}(\tilde{\mathbf{x}}_{1:i});\ \overleftarrow{\mathrm{GRU}}(\tilde{\mathbf{x}}_{i:K})\right] \in \mathbb{R}^{2D}, \qquad \tilde{\mathbf{x}}_i = \mathrm{LN}(\mathbf{x}_i). \tag{4}$$

Because the entire input falls within $[0, T_o)$, the backward pass sees only events already observed at prediction time.

The model combines the hidden states through additive attention [45], used as an attention-pooling mechanism [46]:

$$s_i = \mathbf{w}_2^\top \tanh(\mathbf{W}_1\mathbf{h}_i + \mathbf{b}_1) + b_2, \qquad \alpha_i = \frac{\exp(s_i)}{\sum_{k=1}^{K}\exp(s_k)}, \qquad \mathbf{c} = \sum_{i=1}^{K}\alpha_i\mathbf{h}_i. \tag{5}$$

Here, $\mathbf{W}_1 \in \mathbb{R}^{64\times 128}$. Padded positions are masked before the softmax calculation. Pooling requires $O(K)$ operations per cascade, whereas full self-attention over the sequence would require $O(K^2)$ operations [47]. The attention weights α_i provide an indication of which observed events contribute most to the final representation.

A bottleneck regression head maps the context vector $\mathbf{c}$ to the predicted target:

$$\hat{y} = \mathbf{w}_5^\top \mathrm{ReLU}\big(\mathbf{W}_4\,\mathrm{Dropout}(\mathrm{ReLU}(\mathbf{W}_3\mathbf{c} + \mathbf{b}_3)) + \mathbf{b}_4\big) + b_5. \tag{6}$$

Its layer widths are $128 \to 64 \to 32 \to 1$, with ReLU activations and dropout at rate 0.3 [48]. The output bias b_5 is initialised to the training-set mean of y, so the initial model prediction corresponds to the mean predictor.

Table 2. Cascade statistics after preprocessing. The average number of events per cascade is calculated on the training split.

T_o	Train	Val	Test	Total	Mean events/cascade
15 min	16,509	3,538	3,538	23,585	45.3
1 h	29,530	6,328	6,328	42,186	61.7

4.4 Training Objective

Training minimises the mean squared error between the prediction $\hat{y}$ and $y = \log_2(\Delta P + 1)$. This objective is the same as the MSLE defined in Eq. (2), so the training loss directly matches the primary evaluation metric. The complete model has 49,824 trainable parameters: 31,104 in the bidirectional GRU, 8,321 in the attention scorer, 10,369 in the regression head, and 30 in input normalisation.

5 Experimental Setup

5.1 Dataset and Preprocessing

We use the Sina Weibo dataset released with DeepHawkes. It contains 119,313 messages posted on 1 June 2016, along with timestamped retweet propagation paths [24]. To keep the cascade population consistent with the published benchmark, we apply the filters in the authors' released preprocessing code [24]: messages published outside 08:00–18:59 Beijing time are removed, paths containing the malformed node identifier -1 are discarded, and only cascades with between 10 and 1,000 observed entries are retained.

Cascades are sorted by publication time and divided chronologically into training, validation, and test sets in a 70/15/15 ratio. Each cascade appears in only one split, and all test cascades were published after the training cascades. The target is the increase in cascade size over a 24-hour horizon, so $T_p = 24\,\text{h}$.

We evaluate two observation windows. The one-hour window is the standard setting used in prior work on this dataset [24,27] and serves as the main comparison point. The 15-minute window tests how well the model performs when substantially less early activity is available. Because the minimum-size filter is applied within the observation window, the two settings contain different cascade populations (Table 2).

5.2 Evaluation Metrics

The primary metric is MSLE in $\log_2$ units, as defined in Eq. (2). We also report MSLE in natural-log units, the coefficient of determination (R^2), and the Pearson correlation coefficient (PCC) between predictions and targets in $\log_2$ space. On raw retweet counts, we report mean absolute error (MAE) and median absolute error (MedAE). The comparison with CasSubTS additionally includes mean absolute percentage error (MAPE), following the definition in [27] and expressed as a fraction.

5.3 Compared Models

Baselines.

- **DeepHawkes** [24]. This recurrent model operates on retweet paths and retains user-influence, self-excitation, and time-decay factors from Hawkes processes. We use the one-hour MSLE reported in the original paper. Although our preprocessing follows the filters in its released code, we did not rerun DeepHawkes on our splits.
- **CasSubTS** [27]. This model combines temporal and structural information through subgraph sampling, multi-head graph attention, and a bidirectional GRU. We trained it on the same one-hour training, validation, and test splits as CasMotif, using the same three seeds (42, 43, and 44), and report the mean across those runs.
- **Mean predictor.** This baseline predicts the training-set mean of y for every cascade. It is the floor that any learned model must beat.

Ablation variants.

- **Mean-pool and max-pool.** These variants apply the same LayerNorm and regression head to the element-wise mean or maximum of the 15-dimensional event vectors. They omit the recurrent encoder and attention and each contain 3,167 parameters.
- **CasMotif with attentive pooling.** This is the complete model described in Sect. 4, with 49,824 trainable parameters.

5.4 Implementation Details

All models are implemented in PyTorch [49] and trained with AdamW [50]. We use a learning rate of 10^{-3}, weight decay of 10^{-4}, and batch size of 64. Gradients are clipped to a maximum norm of 1.0 [51]. If validation MSLE does not improve for three epochs, the learning rate is halved. Training runs for at most 60 epochs and stops early after eight epochs without validation improvement; the checkpoint with the lowest validation MSLE is restored.

To reduce padding, batches group cascades of similar lengths, and variable-length sequences are packed so that the GRU skips padded steps. Both the feature scaler and the mean-predictor constant are fitted using the training split only. The one-hour comparison with CasSubTS (Table 3) averages three seeds (42, 43, and 44) for each model. All remaining experiments, including the ablations, use seed 42. The seed-42 CasMotif runs stopped at epoch 28 for the 15-minute window (best epoch 20) and epoch 18 for the one-hour window (best epoch 10).

Table 3. Test results on Sina Weibo at $T_o = 1\,\text{h}$ and $T_p = 24\,\text{h}$ (42,186 cascades; chronological 70/15/15 split). CasSubTS and CasMotif are averaged over seeds 42, 43, and 44. MSLE is reported in $\log_2$ units; MAE is calculated on raw retweet counts. *The DeepHawkes result is taken from its original paper [24] and was not rerun.

Model	MSLE ↓	MAPE ↓	MAE ↓	Params
DeepHawkes* [24]	2.4410	–	–	–
CasSubTS [27]	1.9864	**0.2308**	**75.97**	153,970
CasMotif (ours)	**1.9845**	0.2315	77.23	**49,824**

6 Results

6.1 Comparison at the Standard One-Hour Window

Table 3 compares CasMotif with the available baselines at the standard one-hour observation window. CasMotif and CasSubTS are evaluated on the same 42,186 cascades and use identical data splits.

Across three seeds, CasMotif achieves a mean MSLE of 1.9845, compared with 1.9864 for CasSubTS—a difference of approximately 0.1%. A paired t-test finds no statistically significant difference ($p = 0.93$; 95% confidence interval for the difference: $[-0.040, +0.039]$). CasSubTS has slightly lower MAPE (0.2308 compared with 0.2315, a 0.3% difference) and MAE (75.97 compared with 77.23, a 1.7% difference).

CasMotif matches this MSLE with 49,824 parameters against 153,970 for CasSubTS, a 3.1-fold reduction, and without subgraph sampling or graph attention. Relative to the MSLE of 2.441 reported for DeepHawkes [24], CasMotif's MSLE is 18.7% lower. Since DeepHawkes was not rerun under our exact experimental protocol, this is a comparison with the published result rather than a controlled head-to-head evaluation. Overall, the evidence supports *parity* with CasSubTS at a smaller model size; it does not establish that CasMotif is more accurate than CasSubTS.

6.2 Ultra-Early Prediction at 15 Minutes

With only 15 minutes of observed activity, CasMotif reduces MSLE from 5.756 for the mean predictor to 2.441 across 23,585 cascades (CasMotif rows of Table 4). This corresponds to a 57.6% reduction in error relative to the mean baseline. The model explains 57.6% of the variance in the log increment ($R^2 = 0.576$), with a Pearson correlation of 0.761. For raw retweet counts, the median absolute error decreases from 51.9 to 32.0 retweets.

Reducing the observation window from one hour to 15 minutes removes 75% of the available observation time. In the seed-42 results, CasMotif's MSLE increases by 23.3%, from 1.980 to 2.441, while R^2 falls from 0.625 to 0.576. Part of this increase reflects the different cascade populations retained by the two windows: the mean predictor's MSLE is also higher at 15 minutes (5.756 compared

Table 4. Aggregation ablation on Sina Weibo ($T_p = 24\,\mathrm{h}$). MSLE in $\log_2$ and natural-log units, R^2, and PCC are calculated in log space; MAE and MedAE are calculated on raw retweet counts. The best learned model per window is shown in bold. All values are from a single run using seed 42.

T_o	Model	Params	$\mathrm{MSLE}_2 \downarrow$	$\mathrm{MSLE}_e \downarrow$	$R^2 \uparrow$	PCC $\uparrow$	MAE $\downarrow$	MedAE $\downarrow$
15 min	Mean predictor	0	5.756	2.766	0.000	–	267.3	51.9
	Max-pool	3,167	2.910	1.398	0.494	0.704	219.5	33.9
	Mean-pool	3,167	2.776	1.334	0.518	0.723	211.8	**31.9**
	CasMotif	49,824	**2.441**	**1.173**	**0.576**	**0.761**	**205.2**	32.0
1 h	Mean predictor	0	5.283	2.538	0.000	–	105.3	21.5
	Max-pool	3,167	2.230	1.072	0.578	0.761	81.5	11.7
	Mean-pool	3,167	2.083	1.001	0.606	0.778	**76.1**	**11.6**
	CasMotif	49,824	**1.980**	**0.951**	**0.625**	**0.793**	77.8	12.5

with 5.283). Because R^2 normalises against target variance, it provides a more useful cross-window reference than MSLE alone.

We did not find published results for this dataset using an observation window shorter than 30 minutes [29]. The 15-minute evaluation should therefore be treated as an additional reference point for future work, rather than as a direct comparison with previously reported results.

7 Ablation Study

Table 4 compares three ways of aggregating the same causal event sequence: attention pooling over the BiGRU states, mean pooling, and max pooling. All three variants receive the same 15-dimensional event vectors and use the same regression head.

Encoder ablation. The mean- and max-pooling variants retain the event features and regression head but replace the recurrent encoder and attention with permutation-invariant summaries. At 15 minutes, these variants produce MSLE values of 2.776 and 2.910, respectively, compared with 2.441 for CasMotif. Their errors are therefore 13.7% and 19.2% higher than the full model's error. At one hour, the corresponding MSLE values are 2.083 and 2.230, compared with 1.980 for CasMotif—increases of 5.2% and 12.6%.

The same ranking appears on the validation set: at 15 minutes, MSLE is 2.432 for CasMotif, 2.726 for mean pooling, and 3.193 for max pooling; at one hour, the values are 2.007, 2.183, and 2.383, respectively, so the ordering is not an artefact of the test split.

Two observations stand out. First, the order in which motif-annotated events occur carries information beyond the overall composition of the sequence: the pooled models receive exactly the same event vectors but discard their order. Second, the gap between the full model and the pooling variants is larger at

15 minutes. A likely explanation is that with fewer events a simple summary discards a larger share of the signal, although our experiments do not isolate this mechanism.

Pooled features remain informative. Even without sequence modelling, the mean-pooling variant, which has only 3,167 parameters, achieves $R^2 = 0.518$ at 15 minutes and $R^2 = 0.606$ at one hour. The causal event descriptors therefore already capture much of the predictable variation, and the recurrent attentive encoder adds to it.

Error profile. The results on raw counts are more mixed. The pooling variants have competitive median absolute errors, and at one hour the mean-pooling model has a slightly lower MAE than CasMotif (76.1 compared with 77.8). CasMotif's advantage is clearest on the log-space metrics that it is trained to optimise. Those metrics place more emphasis on relative errors across the popularity range, whereas absolute-error measures are strongly affected by the heavy tail of retweet counts. This pattern is consistent with the model's MSLE training objective.

Observation-window ablation. Shortening the window from one hour to 15 minutes reduces R^2 by 0.049 for CasMotif (from 0.625 to 0.576), compared with reductions of 0.088 for mean pooling (0.606 to 0.518) and 0.083 for max pooling (0.578 to 0.494). Among these three variants, the full model therefore shows the smallest decrease in R^2 when less information is available.

8 Limitations

This study has several limitations. First, only the one-hour comparison is averaged across three random seeds. The 15-minute results and the ablation experiments are based on a single seed, so their variability across runs has not been measured. All experiments also use one dataset, which limits conclusions about how well the method transfers to other platforms. Second, the DeepHawkes score is taken from the original publication rather than reproduced on our exact splits. Third, we have not yet performed feature-group or per-motif ablations. Consequently, the separate contributions of motif indicators, reactivation signals, motif latency, timing features, and descendant counts remain unquantified. Finally, the released Sina Weibo data do not include message text, so the model does not use content-based signals.

9 Conclusion

We introduced CasMotif, a method that represents an early information cascade as an ordered sequence of reshare events. Each event is annotated with the local motifs it completes, alongside structural, temporal, and user-reactivation descriptors computed from the graph state immediately before the event is added.

A compact bidirectional GRU with additive attention then maps the sequence to the log-transformed increment in popularity.

On the Sina Weibo benchmark, using the preprocessing supplied with DeepHawkes, CasMotif achieves a three-seed mean $\log_2$ MSLE of 1.985 at the standard one-hour observation window. Its result is statistically indistinguishable from CasSubTS trained on the same splits ($p = 0.93$), while CasMotif uses 3.1 times fewer parameters. When the observation window is reduced to 15 minutes, the model achieves $R^2 = 0.576$. The ablation results further indicate that recurrent attention pooling consistently outperforms order-independent pooling of the same features, with the largest difference in the shorter observation window.

These findings suggest that explicit causal motif features can provide a useful representation of early cascade activity, and that modelling the sequence of those events can improve prediction over simple pooling. The results should nevertheless be interpreted in light of the single-dataset evaluation, the limited multi-seed testing outside the one-hour comparison, and the absence of feature-level ablations.

9.1 Future Work

We see four directions for extending this study.

- **Feature-group and per-motif ablations.** Future experiments will compare the motif block with timing descriptors (including motif latency Δt^M and inter-arrival gaps) and descendant counts, and measure the individual contribution of the FORK, CHAIN, DYAD, and CYCLE indicators.
- **Additional baselines under a common protocol.** We plan to train DeepCas [23], DeepHawkes [24], CasCN [25], CasSeqGCN [35], CasFlow [26], CTCP [28], and HierCas [29] on the same 42,186-cascade chronological splits used in Table 3. This would allow more direct comparisons with the existing controlled CasSubTS evaluation.
- **Evaluation at additional horizons.** Beyond 15 minutes and one hour, we will test observation windows of two and three hours, which are also considered in the CasSubTS work [27]. These experiments will show whether the compact model maintains comparable performance as longer event sequences become available.
- **Additional cascade datasets.** We will evaluate CasMotif on the DBLP citation cascades used by CasSubTS [27] and on Twitter hashtag cascades [11]. These tests will help determine whether the retweet-tree motif representation transfers to other platforms and diffusion settings.

Acknowledgements. We thank Dayanand V of the School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, for guidance throughout this work and for valuable input on its direction, methodology, and presentation. We also thank Cao et al. for releasing the Sina Weibo cascade dataset and DeepHawkes preprocessing code [24], and Xu et al. for making the dataset available through the CCGL mirrors [34]. We are grateful to the authors of CasSubTS [27] for

documenting their model and evaluation protocol in enough detail to support a controlled comparison. We also acknowledge the developers of PyTorch [49] and the Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Coimbatore, for supporting this work.

References

1. Zhou, F., Xu, X., Trajcevski, G., Zhang, K.: A survey of information cascade analysis: Models, predictions, and recent advances. ACM Computing Surveys **54**(2), 27:1–27:36 (2021)
2. Leskovec, J., Adamic, L.A., Huberman, B.A.: The dynamics of viral marketing. ACM Transactions on the Web **1**(1), 5 (2007)
3. Kempe, D., Kleinberg, J., Tardos, É.: Maximizing the spread of influence through a social network. In: Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pp. 137–146 (2003)
4. Vosoughi, S., Roy, D., Aral, S.: The spread of true and false news online. Science **359**(6380), 1146–1151 (2018)
5. Gao, X., Cao, Z., Li, S., Yao, B., Chen, G., Tang, S.: Taxonomy and evaluation for microblog popularity prediction. ACM Transactions on Knowledge Discovery from Data **13**(2), 1–40 (2019)
6. Salganik, M.J., Dodds, P.S., Watts, D.J.: Experimental study of inequality and unpredictability in an artificial cultural market. Science **311**(5762), 854–856 (2006)
7. Martin, T., Hofman, J.M., Sharma, A., Anderson, A., Watts, D.J.: Exploring limits to prediction in complex social systems. In: Proceedings of the 25th International Conference on World Wide Web (WWW), pp. 683–694 (2016)
8. Cheng, J., Adamic, L., Dow, P.A., Kleinberg, J.M., Leskovec, J.: Can cascades be predicted? In: Proceedings of the 23rd International Conference on World Wide Web (WWW), pp. 925–936 (2014)
9. Szabo, G., Huberman, B.A.: Predicting the popularity of online content. Communications of the ACM **53**(8), 80–88 (2010)
10. Shulman, B., Sharma, A., Cosley, D.: Predictability of popularity: Gaps between prediction and understanding. In: Proceedings of the 10th International AAAI Conference on Web and Social Media (ICWSM), pp. 348–357 (2016)
11. Weng, L., Menczer, F., Ahn, Y.Y.: Virality prediction and community structure in social networks. Scientific Reports **3**, 2522 (2013)
12. Weng, L., Menczer, F., Ahn, Y.Y.: Predicting successful memes using network and community structure. In: Proceedings of the 8th International AAAI Conference on Weblogs and Social Media (ICWSM), pp. 535–544 (2014)
13. Bao, P., Shen, H.W., Huang, J., Cheng, X.Q.: Popularity prediction in microblogging network: A case study on Sina Weibo. In: Proceedings of the 22nd International Conference on World Wide Web (WWW Companion), pp. 177–178 (2013)
14. Goel, S., Anderson, A., Hofman, J., Watts, D.J.: The structural virality of online diffusion. Management Science **62**(1), 180–196 (2016)
15. Hawkes, A.G.: Spectra of some self-exciting and mutually exciting point processes. Biometrika **58**(1), 83–90 (1971)
16. Crane, R., Sornette, D.: Robust dynamic classes revealed by measuring the response function of a social system. Proceedings of the National Academy of Sciences **105**(41), 15649–15653 (2008)

17. Shen, H., Wang, D., Song, C., Barabási, A.L.: Modeling and predicting popularity dynamics via reinforced Poisson processes. In: Proceedings of the 28th AAAI Conference on Artificial Intelligence (2014)
18. Zhao, Q., Erdogdu, M.A., He, H.Y., Rajaraman, A., Leskovec, J.: SEISMIC: A self-exciting point process model for predicting tweet popularity. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pp. 1513–1522 (2015)
19. Gao, S., Ma, J., Chen, Z.: Modeling and predicting retweeting dynamics on microblogging platforms. In: Proceedings of the 8th ACM International Conference on Web Search and Data Mining (WSDM), pp. 107–116 (2015)
20. Rizoiu, M.A., Xie, L., Sanner, S., Cebrian, M., Yu, H., Van Hentenryck, P.: Expecting to be HIP: Hawkes intensity processes for social media popularity. In: Proceedings of the 26th International Conference on World Wide Web (WWW), pp. 735–744 (2017)
21. Yu, L., Cui, P., Wang, F., Song, C., Yang, S.: From micro to macro: Uncovering and predicting information cascading process with behavioral dynamics. In: Proceedings of the IEEE International Conference on Data Mining (ICDM), pp. 559–568 (2015)
22. Mishra, S., Rizoiu, M.A., Xie, L.: Feature driven and point process approaches for popularity prediction. In: Proceedings of the 25th ACM International Conference on Information and Knowledge Management (CIKM), pp. 1069–1078 (2016)
23. Li, C., Ma, J., Guo, X., Mei, Q.: DeepCas: An end-to-end predictor of information cascades. In: Proceedings of the 26th International Conference on World Wide Web (WWW), pp. 577–586 (2017)
24. Cao, Q., Shen, H., Cen, K., Ouyang, W., Cheng, X.: DeepHawkes: Bridging the gap between prediction and understanding of information cascades. In: Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM), pp. 1149–1158 (2017)
25. Chen, X., Zhou, F., Zhang, K., Trajcevski, G., Zhong, T., Zhang, F.: Information diffusion prediction via recurrent cascades convolution. In: Proceedings of the 35th IEEE International Conference on Data Engineering (ICDE), pp. 770–781 (2019)
26. Xu, X., Zhou, F., Zhang, K., Liu, S., Trajcevski, G.: CasFlow: Exploring hierarchical structures and propagation uncertainty for cascade prediction. IEEE Transactions on Knowledge and Data Engineering **35**, 3484–3499 (2023)
27. Wang, J., Li, J., Liu, C., Peng, Y., Li, X.: Fusing temporal and structural information via subgraph sampling and multi-head attention for information cascade prediction. Scientific Reports **15**, 6801 (2025)
28. Lu, X., Ji, S., Yu, L., Sun, L., Du, B., Zhu, T.: Continuous-time graph learning for cascade popularity prediction. In: Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI), pp. 2224–2232 (2023)
29. Zhang, Z., Xie, X., Zhang, Y., Zhang, L., Jiang, Y.: HierCas: Hierarchical temporal graph attention networks for popularity prediction in information cascades. arXiv preprint arXiv:2310.13219 (2023)
30. Milo, R., Shen-Orr, S., Itzkovitz, S., Kashtan, N., Chklovskii, D., Alon, U.: Network motifs: Simple building blocks of complex networks. Science **298**(5594), 824–827 (2002)
31. Kovanen, L., Karsai, M., Kaski, K., Kertész, J., Saramäki, J.: Temporal motifs in time-dependent networks. Journal of Statistical Mechanics: Theory and Experiment **2011**(11), P11005 (2011)

32. Paranjape, A., Benson, A.R., Leskovec, J.: Motifs in temporal networks. In: Proceedings of the 10th ACM International Conference on Web Search and Data Mining (WSDM), pp. 601–610 (2017)
33. Benson, A.R., Gleich, D.F., Leskovec, J.: Higher-order organization of complex networks. Science **353**(6295), 163–166 (2016)
34. Xu, X., Zhou, F., Zhang, K., Liu, S.: CCGL: Contrastive cascade graph learning. IEEE Transactions on Knowledge and Data Engineering **35**(5), 4539–4554 (2023)
35. Wang, Y., Wang, X., Ran, Y., Michalski, R., Jia, T.: CasSeqGCN: Combining network structure and temporal sequence to predict information cascades. Expert Systems with Applications **206**, 117693 (2022)
36. Sun, X., Zhou, J., Liu, L., Wei, W.: Explicit time embedding based cascade attention network for information popularity prediction. Information Processing & Management **60**, 103278 (2023)
37. Wang, J., Zheng, V.W., Liu, Z., Chang, K.C.C.: Topological recurrent neural network for diffusion prediction. In: Proceedings of the IEEE International Conference on Data Mining (ICDM), pp. 475–484 (2017)
38. Kazemi, S.M., Goel, R., Jain, K., Kobyzev, I., Sethi, A., Forsyth, P., Poupart, P.: Representation learning for dynamic graphs: A survey. Journal of Machine Learning Research **21**(70), 1–73 (2020)
39. Kumar, S., Zhang, X., Leskovec, J.: Predicting dynamic embedding trajectory in temporal interaction networks. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pp. 1269–1279 (2019)
40. Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., Achan, K.: Inductive representation learning on temporal graphs. In: International Conference on Learning Representations (ICLR) (2020)
41. Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., Bronstein, M.: Temporal graph networks for deep learning on dynamic graphs. arXiv preprint arXiv:2006.10637 (2020)
42. Ba, J.L., Kiros, J.R., Hinton, G.E.: Layer normalization. arXiv preprint arXiv:1607.06450 (2016)
43. Schuster, M., Paliwal, K.K.: Bidirectional recurrent neural networks. IEEE Transactions on Signal Processing **45**(11), 2673–2681 (1997)
44. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1724–1734 (2014)
45. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. In: International Conference on Learning Representations (ICLR) (2015)
46. Yang, Z., Yang, D., Dyer, C., He, X., Smola, A., Hovy, E.: Hierarchical attention networks for document classification. In: Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), pp. 1480–1489 (2016)
47. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008 (2017)
48. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research **15**(56), 1929–1958 (2014)

49. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al.: PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32, pp. 8024–8035 (2019)
50. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2019)
51. Pascanu, R., Mikolov, T., Bengio, Y.: On the difficulty of training recurrent neural networks. In: Proceedings of the 30th International Conference on Machine Learning (ICML), pp. 1310–1318 (2013)